SAFL Secure AI Futures Lab

India Agentic AI Security Hackathon: Submission Template

Secure AI Futures Lab | August - October 2026

1. Team information

Field	Your answer
Team name	Root Coders
Primary contact name	Madhu S
Primary contact email (the one you registered with)	madhu.s.3606@gmail.com
Primary contact phone / WhatsApp (with country code)	+91 9750595939
Team size (1, 2 or 3)	1

2. Team members

Fill one column per member, up to three. Write names as they should appear if we credit you.

Field	Member 1 (primary contact)
Full name	Madhu S

Email	madhu.s.3606@gmail.com
Institution / organisation ("Independent" if none)	East West Institute of Technology
Background: student / engineer or researcher/ policy, legal or economics professional / other (specify)	Student

3. How you took part

Field	**Your answer**
Participation mode: IIT Delhi Jam Site (Aug) / Bengaluru Jam Site (17 Oct) / online only	Online only
Track: Technical (working prototype or technical research) / Non-tech/Policy (policy brief or paper)	Working prototype
Challenge area (pick the one your work fits best; "Open category" if it fits none of the five anchored areas): Trust & Consent / Accountability & Liability / System Resilience / Inclusive Design / Fair Markets / Open category. Area descriptions: Tracks section of the hackathon page	Trust & Consent

4. Project overview

Project title

Kavach: Cryptographic Micro-Mandates for Agentic AI

One-line summary

Cryptographic micro-mandates that restrict an autonomous AI agent's UPI spending authority to user-approved payees, amounts, and budgets, preventing prompt-injection fund draining.

Website description

Autonomous AI agents operating on India's Digital Public Infrastructure can be hijacked by indirect prompt injections in merchant catalogs or invoices to divert payments to attacker accounts. Kavach addresses this by isolating payment credentials from the agent and enforcing asymmetric cryptographic micro-mandates via an out-of-process interceptor. Every transaction requires an Ed25519-signed authorization verifying allowlisted payees, per-transaction limits, cumulative budgets, and anti-replay nonces. In empirical benchmarks, Kavach blocked 10/10 direct payment attacks and withstood 70 adaptive LLM adversarial attempts with zero bypasses, securing delegated financial agency on UPI and ONDC.

5. The problem

Which DPI system(s) does your work address?

UPI/ONDC

Which DPI assumption breaks when AI agents act on it?

India's Unified Payments Interface (UPI) assumes that a human user consciously reviews, validates, and authorizes each payment transaction before entering their UPI PIN. When an autonomous AI agent acts as a financial delegate—such as autonomously purchasing groceries on ONDC or settling utility bills—this human-in-the-loop validation is removed. Because language models are vulnerable to indirect prompt injections planted in untrusted third-party data like product descriptions, web listings, or supplier invoices, an attacker can hijack the agent's decision logic. When the agent initiates an API call to a payment gateway, the system treats it as an authorized user request. The core DPI assumption that payment authorization originates from uncoerced human intent breaks completely, turning delegated authority into automated financial exfiltration.

Problem statement and who is affected

As consumers and small merchants adopt autonomous AI agents for automated procurement on ONDC and UPI, an attacker can embed hidden instructions in invoice PDFs, product titles, or digital catalogs. When an agent processes this data, the injection overrides its goals, directing it to disburse funds to an attacker's Virtual Payment Address (VPA) or exhaust balances through inflated purchases. Small merchants and individual consumers bear the financial losses because payment switches cannot distinguish an injection-commandeered agent API call from legitimate user activity. Because UPI processes over 24 billion monthly transactions and agent designs are replicated across thousands of users, a single weaponized catalog entry can systematically drain funds across hundreds of autonomous agents simultaneously before detection occurs, undermining public trust in agentic digital commerce

6. Your approach and findings

We built Kavach, a fail-closed, multi-process authorization architecture that strips AI agents of direct payment credentials and bounds their spending using cryptographic micro-mandates. Before an agent executes tasks, the user signs an asymmetric Ed25519 micro-mandate token encoding explicit constraints: allowlisted payee VPAs, allowed product categories, per-transaction ceilings, cumulative budgets, and expiry.

The agent runs in an untrusted layer holding zero cryptographic keys or payment tokens, interacting only with typed tools. All payment calls route through an isolated local interceptor service. The interceptor executes an ordered 11-step fail-closed policy evaluation: verifying the Ed25519 signature, algorithm pinning (EdDSA only), time validity, revocation status, exact payee allowlists, non-zero positive integer paise amounts, per-transaction caps, and maximum transaction counts. It enforces cumulative budget tracking with atomic two-phase reservations (reserve then commit or rollback) via SQLite BEGIN IMMEDIATE to prevent race conditions. Upon approval, the interceptor issues an ephemeral, single-use Ed25519 ApprovalTicket (30-second expiry) bound to the payee, amount, mandate identifier, and a unique anti-replay nonce. The simulated UPI gateway settles payments only when presented with a valid ticket, rejecting direct agent requests. High-value transactions trigger an interactive What-You-See-Is-What-You-Sign step-up consent challenge.

Threat model and resilience under exposure

The attacker controls untrusted third-party inputs that the agent ingests (such as ONDC product listings, web search results, or invoices) and attempts to steal funds by redirecting payments to an unauthorized UPI VPA or exhausting user balances. We assume full Kerckhoffs-style exposure: the attacker has complete knowledge of the system prompt, tool schemas, policy rules, source code, interceptor port, and model weights.

Kavach maintains resilience under exposure because enforcement does not rely on secrecy, prompt engineering, or LLM-based behavioral classification. The interceptor is an out-of-process deterministic proxy that evaluates cryptographic invariants mathematically. Even if an attacker completely commandeers the agent's internal reasoning via jailbreak or prompt injection:

1. The agent cannot pay an unauthorized payee because the interceptor enforces strict byte-level allowlist validation on destination VPAs.

2. The agent cannot exceed spending limits because integer-paise arithmetic and atomic ledger reservations reject any transaction exceeding per-transaction or cumulative budget caps.

3. The agent cannot forge approval tickets because it holds zero private keys.

4. The agent cannot replay prior tickets because nonces are single-use.

The remaining attack surface within our threat model is in-mandate abuse: an attacker could trick an agent into buying unintended items from an authorized payee up to the remaining mandate budget.

How you evaluated it

We evaluated Kavach across three tiers of empirical testing using an end-to-end simulated ONDC grocery shopping environment and a mock UPI gateway:

1. **Default Live LLM Benchmark ($N = 100$):** Evaluated llama3.1:latest (8B) via Ollama across 5 indirect prompt injection archetypes (direct imperative, fake system override, fake refund notice, Hindi/Hinglish, and JSON-nested payload) ×10 trials under Protection OFF vs. ON, with 100% catalog exposure guarantee.

2. **Positive Control ($N = 20$):** Evaluated the test harness by directly instructing the agent to transfer funds to an attacker wallet under Protection OFF vs. ON.

3. **Adaptive LLM Red-Teaming ($N = 70$):** Deployed an adversarial LLM agent with full knowledge of Kavach's defense architecture, generating 70 attack requests across 7 attack vectors (payee substitution, overspending, count exhaustion, budget exhaustion, replay, argument smuggling, and category mismatch).

4. **Formal Invariant Suite**: Ran 137 property-based and crash resilience tests (pytest and Hypothesis, 1,000 derandomized iterations per invariant in CI).

5. **Latency Benchmarks**: Measured 1,000 requests for in-process policy evaluation and full two-hop HTTP RPC latency.

Key results

- **Positive Control**: Without Kavach, 10/10 direct attacks succeeded transferring ₹50.00 each; with Kavach enabled, blocked 10/10 attempts (95% CI 69.2%–100.0%) with ₹0.00 transferred (evidence/live/positive_control_transcripts.jsonl).
- **Adaptive Attacker Resistance**: 70 LLM-driven adversarial attempts yielded 0 bypasses (95% upper bound 5.1%). 56 were wrong-payee attempts, 10 probed in-mandate limits (3 approved within budget, 5 BUDGET_EXHAUSTED, 2 AMOUNT_EXCEEDS_PER_TXN_LIMIT), 2 CATEGORY_NOT_ALLOWED, and 2 timeouts.
- **Live Injection Exposure**: In 100 live LLM trials across 5 injection archetypes, 0 of 100 trials obeyed the injection (50 OFF, 50 ON). In the JSON-payload condition, in 8/10 trials the model replied with text and made no tool call. Table 2 (weakened-guard) was an invalid run due to a harness defect omitting tool instructions and is excluded from claims.
- **Microsecond Enforcement Latency**: In-process policy check p50 = 1.09 ms, p99 = 3.61 ms (evidence/latency.txt); end-to-end two-hop HTTP RPC p50 = 7.08 ms (evidence/latency_rpc.txt).

What's new here

Instead of treating prompt injection as an input-filtering problem using probabilistic guardrails, Kavach treats delegated agency as a cryptographic authorization problem. It translates UPI e-Mandates into ephemeral, client-signed Ed25519 micro-mandates with atomic ledger enforcement that hold even when the model is completely compromised.

Who could use this, and how

NPCI and payment aggregators could adopt this architecture by standardizing an "AI Agent Delegate" profile within UPI AutoPay specifications. Banks and Payment Service Providers (PSPs) can embed the lightweight Ed25519 ticket verification logic directly into acquiring switches to validate agent payments without retraining or modifying upstream LLMs.

Limitations and open problems

Kavach operates within defined boundaries:

1. **In-Mandate Abuse**: An attacker cannot redirect payments to third parties, but can induce unintended purchases to an authorized merchant up to the remaining mandate budget. Mitigating price-gouging by compromised authorized sellers requires signed merchant quotes, which is currently design-only.

2. **Denial of Service**: A hijacked agent can exhaust transaction counts or budgets on small items, starving legitimate tasks.

3. **Prototype Environment**: Evaluated on local mock payment switches and simulated ONDC stores, not live banking production networks.

4. **Key Management**: Prototype uses filesystem Ed25519 PEM keys; production requires Hardware Security Modules (HSM) or device Secure Enclaves.

5. **Container Isolation**: Docker isolation was partially verified in CI (interceptor reachable, gateway unreachable, no keys in env); filesystem read-only and non-root user were not asserted in CI.

7. Reflection

Difficulty faced

The hardest challenge we encountered was resolving the fundamental tension between autonomous agent utility and deterministic delegation safety. Our earliest prototype attempted to validate tool calls by having a secondary "verifier" LLM inspect the agent's planned action against the user's natural language prompt. In initial experiments, this approach failed completely: the verifier LLM was vulnerable to the exact same

indirect prompt injections that compromised the shopping agent, sometimes hallucinating that an attacker's wallet was an authorized payment recipient.

When we replaced the LLM verifier with strict string matching against user instructions, we encountered the opposite failure mode: false blocks broke agent utility. In an open commerce network like ONDC, seller identifiers and catalog prices vary dynamically. If a user provided a conversational instruction like "order groceries under ₹1,000", a strict string check could not verify whether a specific vendor VPA like ondc-grocer@upi or an item price of ₹670 was authorized.

This exposed a critical insight: **natural language intent is inherently underspecified and cannot serve as a runtime security boundary**. Expecting an agent or a model-based guardrail to infer security boundaries dynamically invites exploitation. The tension forced us to recognize that financial delegation requires a clear separation of concerns: users must establish explicit, structured boundaries before delegation begins (specifying allowlisted payees, spending caps, and valid categories), while agents retain the freedom to infer operational parameters (item selection, quantities, notes) within those cryptographic boundaries. This shifted our architecture entirely from semantic analysis to deterministic cryptographic enforcement.

How did you overcome it?

- **Decoupled Strategic Consent from Tactical Execution**: Mandated that users explicitly sign structured parameters (allowlisted payee VPAs, maximum per-transaction amounts, total budget, and category) at mandate creation time, while permitting the agent to autonomously select cart items and notes within that envelope.

- **Replaced Probabilistic LLM Verifiers with Deterministic Cryptography**: Eliminated model-based filtering in favor of an out-of-process interceptor executing 11 fail-closed checks and issuing Ed25519-signed single-use approval tickets, giving up open-ended unstructured payment flexibility to achieve unbypassable security under adversarial exposure.

- **Implemented Two-Phase Atomic Concurrency Reservations**: Prevented Time-of-Check to Time-of-Use (TOCTOU) race conditions in multi-transaction spending using SQLite BEGIN IMMEDIATE locks (reserve then commit/rollback).

- **Designed Step-Up Consent for Ambiguous High-Value Edge Cases**: For transactions exceeding preset thresholds, implemented an out-of-band interactive WYSIWYS preview with SHA-256 fingerprinting (ESCALATE challenge).

- **Still Open**: Protection against colluding authorized merchants who inflate prices within mandate caps remains an open problem (documented as design-only signed merchant quotes in docs/MERCHANT_QUOTES.md).

8. Deliverables and declarations

Field	Guidance	Your answer
Main submission file	Root_Coders_report.pdf	Example: https://drive.google.com/.../CyberGuardians_Report.pdf
Code repository	Include a README explaining how to run code and reproduce results	https://github.com/Madhu-36/Kavach
Demo video	Max 2 minutes	https://youtu.be/2q6kAZHDG0k
Other artifacts (optional)	Datasets, scenario sets, slides	evidence/ directory (SHA-256 manifest in evidence/MANIFEST.sha256), walkthrough.md, evaluation benchmark transcripts in evidence/live/
Was this work done during the hackathon?	Yes / Built on earlier work. If you built on earlier work by your team, say what existed before and what you added during the hackathon	Yes
AI tools used, and what for	List each tool with the exact model or version and its role	Ollama llama3.1:latest (8B model for live injection, positive control, and adaptive attacker evaluation benchmarks), Google Antigravity / Gemini 2.5 Pro (coding assistance, architecture reviews, and test harness development)
External code, models or datasets used	Include licences	cryptography (Apache 2.0 / BSD), PyJWT (MIT), FastAPI & Uvicorn (MIT), Pydantic (MIT), pytest & Hypothesis (MPL 2.0), Meta Llama 3.1 8B (Llama 3.1 Community License)

All links open publicly without a login	Yes / No	Yes

9. Responsible testing

Both declarations must be "Yes" for your submission to be evaluated.

Declaration	**Yes / No**
No real personal data: We did not use real Aadhaar numbers, UPI IDs, bank details or anyone's personal data. Any such data in our work is synthetic.	Yes
Responsible disclosure: Our public materials don't include working exploit details that could put real users at risk. If we found a weakness that may affect a real system, we have shared it privately with SAFL at team@secureaifutureslab.com instead of publishing it, and SAFL will get back to us on next steps.	Yes

Environment you tested in (1-2 lines)

A locally hosted mock UPI payment gateway (:8000), simulated ONDC store catalog, and local Kavach Interceptor service (:8100) running on Python 3.11–3.14, with container isolation verified on GitHub Actions Ubuntu runners.

10. Ownership, permissions and consent

Ownership: Your team keeps full ownership of your work. Submitting gives SAFL permission to use it only in the ways you agree to below.

Always shared, needed for evaluation: SAFL's judges and mentors will see your full submission. We will never publish your email addresses or phone numbers.

#	**Permission**	**Yes / No**	**Members opting out (names)**

1	**Featuring your project:** May SAFL publish your project title, summary, website description and public links, with credit to your team, on the SAFL website, SAFL social media (e.g. LinkedIn, X), newsletters, and hackathon reports or presentations?	Yes	None
2	**Your names and institutions:** May we name you and your institutions when featuring your project in those places?	Yes	None